

Step 2 — A Real Terminal Chat With Memory and Live Streaming — Solution

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Basic Version

Approach 1 — the direct way

```
# chat_client.py
def stream_message(client, messages):
    stream = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=messages,
        stream=True,
    )
    full_reply = ""
    for chunk in stream:
        piece = chunk.choices[0].delta.content
        if piece:
            print(piece, end="", flush=True)
            full_reply += piece
    print()
    messages.append({"role": "assistant", "content": full_reply})
    return messages
```

```
# main.py
from config import load_config
from chat_client import create_client, stream_message

config = load_config()
client = create_client(config.openai_api_key)

messages = []
while True:
    user_text = input("You: ")
    messages.append({"role": "user", "content": user_text})
    messages = stream_message(client, messages)
```

This works and passes the 3-message memory test — the model sees the full history every time because `messages` keeps growing and gets resent whole. It's missing `send_message()` (non-streaming) as a separate option, type hints, and any limit on how large `messages` is allowed to get.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — both functions, typed, with a real exit condition

```
# chat_client.py
from openai import OpenAI

def send_message(client: OpenAI, messages: list[dict]) -> list[dict]:
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=messages,
    )
    reply = response.choices[0].message.content
    messages.append({"role": "assistant", "content": reply})
    return messages

def stream_message(client: OpenAI, messages: list[dict]) -> list[dict]:
    stream = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=messages,
        stream=True,
    )
    full_reply = ""
    for chunk in stream:
        piece = chunk.choices[0].delta.content
        if piece:
            print(piece, end="", flush=True)
            full_reply += piece
    print()
    messages.append({"role": "assistant", "content": full_reply})
    return messages
```

```
# main.py
from config import load_config
from logging_setup import get_logger
from chat_client import create_client, stream_message

config = load_config()
```

```
logger = get_logger(__name__)
client = create_client(config)

messages: list[dict] = []
print("Type 'quit' to exit.")

while True:
    user_text = input("You: ")
    if user_text.strip().lower() == "quit":
        break

    messages.append({"role": "user", "content": user_text})
    logger.debug("Sending %d messages to the model", len(messages))
    print("Assistant: ", end="", flush=True)
    messages = stream_message(client, messages)
```

Difference from Basic: both `send_message()` and `stream_message()` exist, fully typed, so you can choose per-call whether you need live output or just the final text (Step 3's extraction call will actually prefer the non-streaming one). `main.py` has a real exit condition instead of only `Ctrl+C`, and logs the growing message count at `DEBUG`, which is exactly the kind of detail you want in a log file but not cluttering the screen. This version still has no upper bound on how large `messages` gets — that's what Advanced adds.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — trim by an estimated token budget

```
def estimate_tokens(messages: list[dict]) -> int:
    total_chars = sum(len(m["content"]) for m in messages)
    return total_chars // 4 # rough rule of thumb: ~4 characters per token

def trim_history(messages: list[dict], max_tokens: int = 3000) -> list[dict]:
    while estimate_tokens(messages) > max_tokens and len(messages) > 1:
        if messages[0]["role"] == "system":
            del messages[1] # keep the system prompt, drop the oldest real turn
        else:
            del messages[0]
    return messages
```

```
# main.py
messages: list[dict] = []
print("Type 'quit' to exit.")

while True:
    user_text = input("You: ")
    if user_text.strip().lower() == "quit":
        break

    messages.append({"role": "user", "content": user_text})
    messages = trim_history(messages, max_tokens=3000)
    logger.debug("Sending %d messages (~%d tokens) to the model", len(messages),
    print("Assistant: ", end="", flush=True)
    messages = stream_message(client, messages)
```

Expected behavior: a normal-length conversation is unaffected — you won't see anything drop. Force it by setting `max_tokens` very low (like `200`) and having a multi-turn conversation. You'll see older turns quietly disappear from what gets sent, while `messages` itself keeps the most recent ones.

Approach 2 — trim by message count instead

```
def trim_history(messages: list[dict], max_messages: int = 20) -> list[dict]:
    if len(messages) > max_messages:
        overflow = len(messages) - max_messages
        del messages[0:overflow]
    return messages
```

Simpler to write than Approach 1, but cruder. Two unusually long messages — a customer pasting a huge error log, say — could still blow your real token budget, even while you're well under the message-*count* cap. Approach 1 tracks actual content size. Approach 2 only tracks how many turns there have been.

Approach 3 — exact token counts with `tiktoken`

```
import tiktoken

encoding = tiktoken.encoding_for_model("gpt-4o-mini")

def estimate_tokens(messages: list[dict]) -> int:
    return sum(len(encoding.encode(m["content"])) for m in messages)
```

This replaces the `len(text) // 4` guess with the same tokenizer the model itself uses. The count now matches what you're actually being charged for, not an approximation. It costs an extra dependency (`pip install tiktoken`) and a little more compute per check. That's worth it once you're trimming close to a real limit and a rough guess isn't good enough — not worth it for a beginner project where "well under the limit" is good enough.

Difference from Intermediate, and between these 3 Advanced approaches: Intermediate lets `messages` grow with no limit at all, and trusts the conversation never gets close to the model's context window. All three Advanced approaches add a real budget and a policy for going over it. Approach 1 estimates content size with a cheap, rough formula — no new dependency, good enough for this project. Approach 2 is even simpler, but ignores message size entirely, so it can trim too little or too much compared to what the token budget actually needs. Approach 3 is the accurate version — it uses the real tokenizer instead of a guess, at the cost of one more dependency.

Which one should you actually write? Approach 1, for this project. It needs no new dependency, and `len(text) // 4` is close enough when you're trimming well under the real limit rather than right up against it. Reach for Approach 3 (`tiktoken`) once you're building something where getting close to the exact limit actually matters — a system handling long documents or very long conversations, for instance. It's worth knowing about now, even if this project doesn't need it yet.